

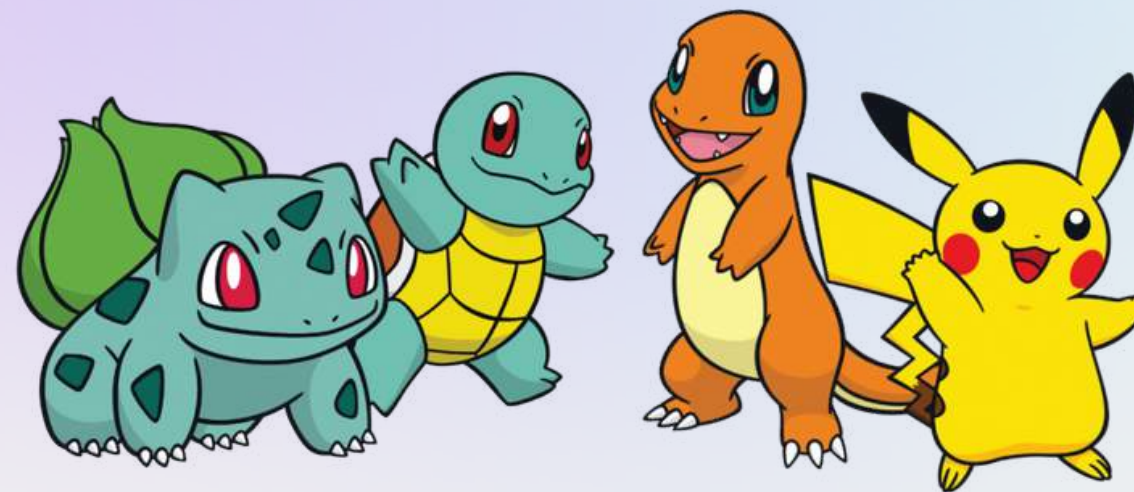
2024/25

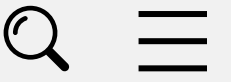
Pokémon The Series

SEARCH SYSTEM

Information Processing and Retrieval

MILESTONE #2





Content



 1. INTRO

 2. DOCUMENTS

 3. SCHEMA AND INDEXING

 4. QUERIES

 5. EVALUATION

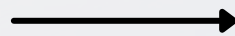
 6. FUTURE WORK

Intro

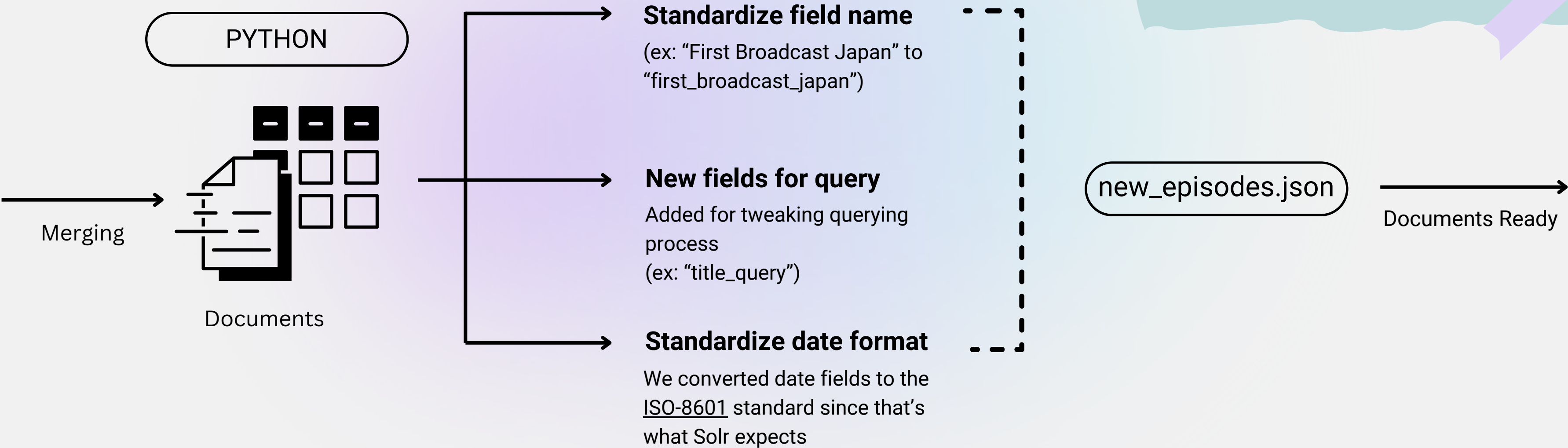
M2 - 4 steps:

- More document processing;
- Designing schemas and indexing fields for 2 systems;
- Making the queries;
- Evaluating the systems.

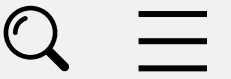




Documents Processing

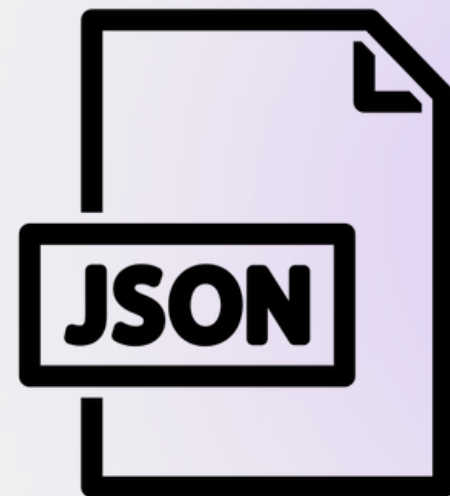


We used Python to process and merge the documents, so that we can make more interesting queries



Schema definition

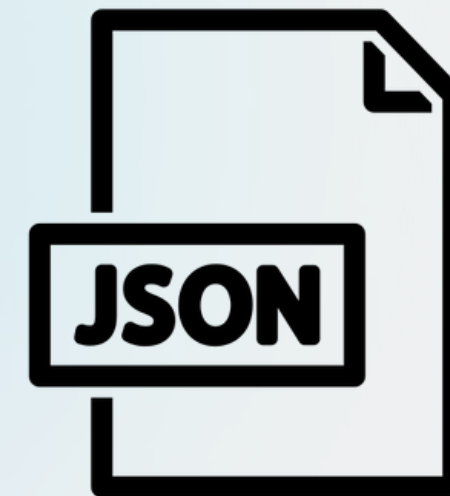
SIMPLE SCHEMA



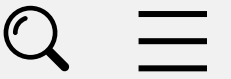
The simple schema contains only the default field types and simple indexes



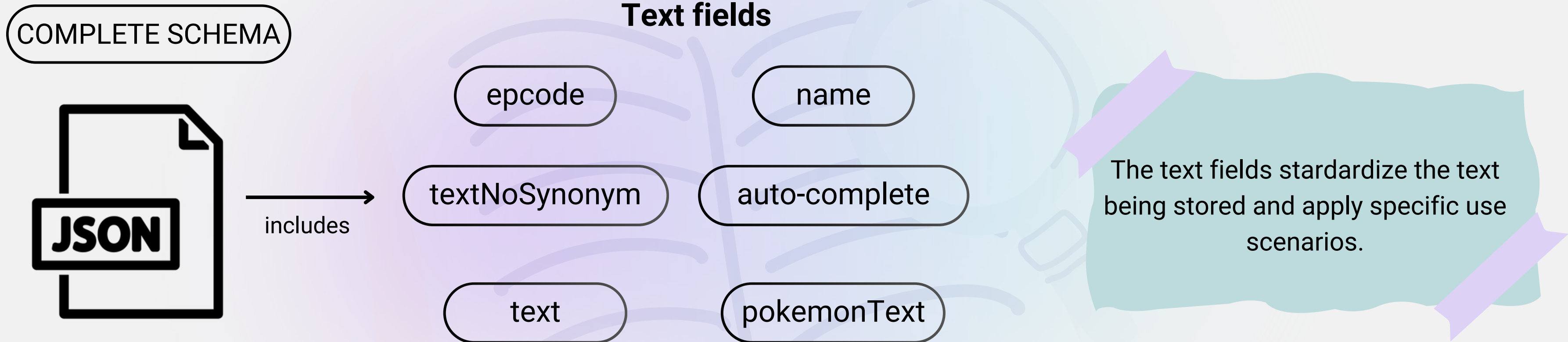
COMPLEX SCHEMA

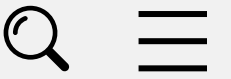


The complex schema has custom field types for better indexing



Schema definition and indexing





Indexed and Query Analyzers

Tokenizers

StandardTokenizerFactory

splits the text the text upon
whitespace or punctuation

EdgeNGramTokenizerFactory

forms n-grams starting at the
edge of the string

Filters

ASCIIFoldingFilterFactory

replaces special characters with
their ASCII counterpart

PorterStemFilterFactory

applies stemming to words
(change, changing -> chang)

SynonymGraphFilterFactory

expand words to include their
synonyms, based on synonyms.txt

Synonyms are only
used at index time, in
text and pokemonText

LowerCaseFilterFactory

uppercase to lowercase characters

BeiderMorseFilterFactory

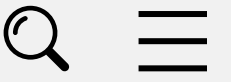
phonetic matching algorithm
(pikashu -> pikachu)

FlattenGraphFilterFactory

must be used after graph-based
filters applied to indexes

EdgeNgramFilterFactory

turn words in N-Grams in a range
of 1 to N without permutations
(Hey->H,He,Hey)



Querying

Our informational needs:

- Episodes involving stealing Pokémon
query: steal pokémon
- Episodes involving Ash winning gym battles (or badges);
query: ash wins gym battles
- Episodes involving Pokémon mega evolutions;
query: mega evolve
- Episodes involving changes to Ash's adventure party;
query: (leave OR join) group
- Pokémon with an angry nature (nested document);
query: angry



Querying

For a better refined search system:



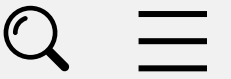
- eDisMax
- Introduction of phrasal slops;
- Selection of relevant fields;
- Use of field boosting;



```
"query": "steal pokemon",
"filter": "-_nest_path:*",
"fields": "id, score",
"limit": 30,
"params": {
  "defType": "edismax",
  "q.op": "AND",
  "qf": "title major_events plot"
}
```



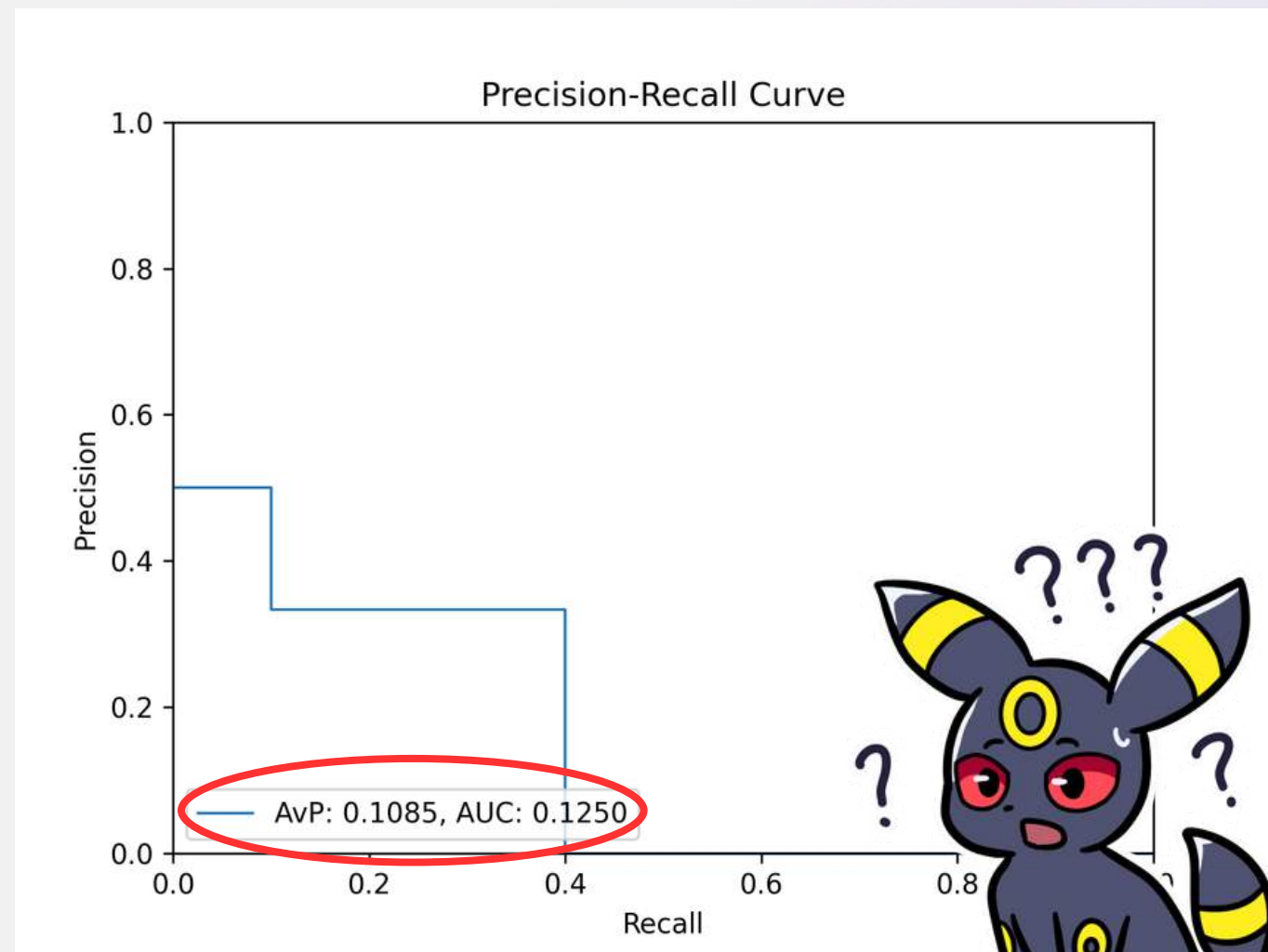
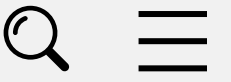
```
"query": "steal pokemon",
"filter": "-_nest_path:*",
"fields": "id, score",
"limit": 30,
"params": {
  "defType": "edismax",
  "q.op": "AND",
  "qf": "title_query^7 major_events^10 plot^4",
  "pf": "plot^25 major_events^25",
  "ps": "6"
}
```



Evaluation

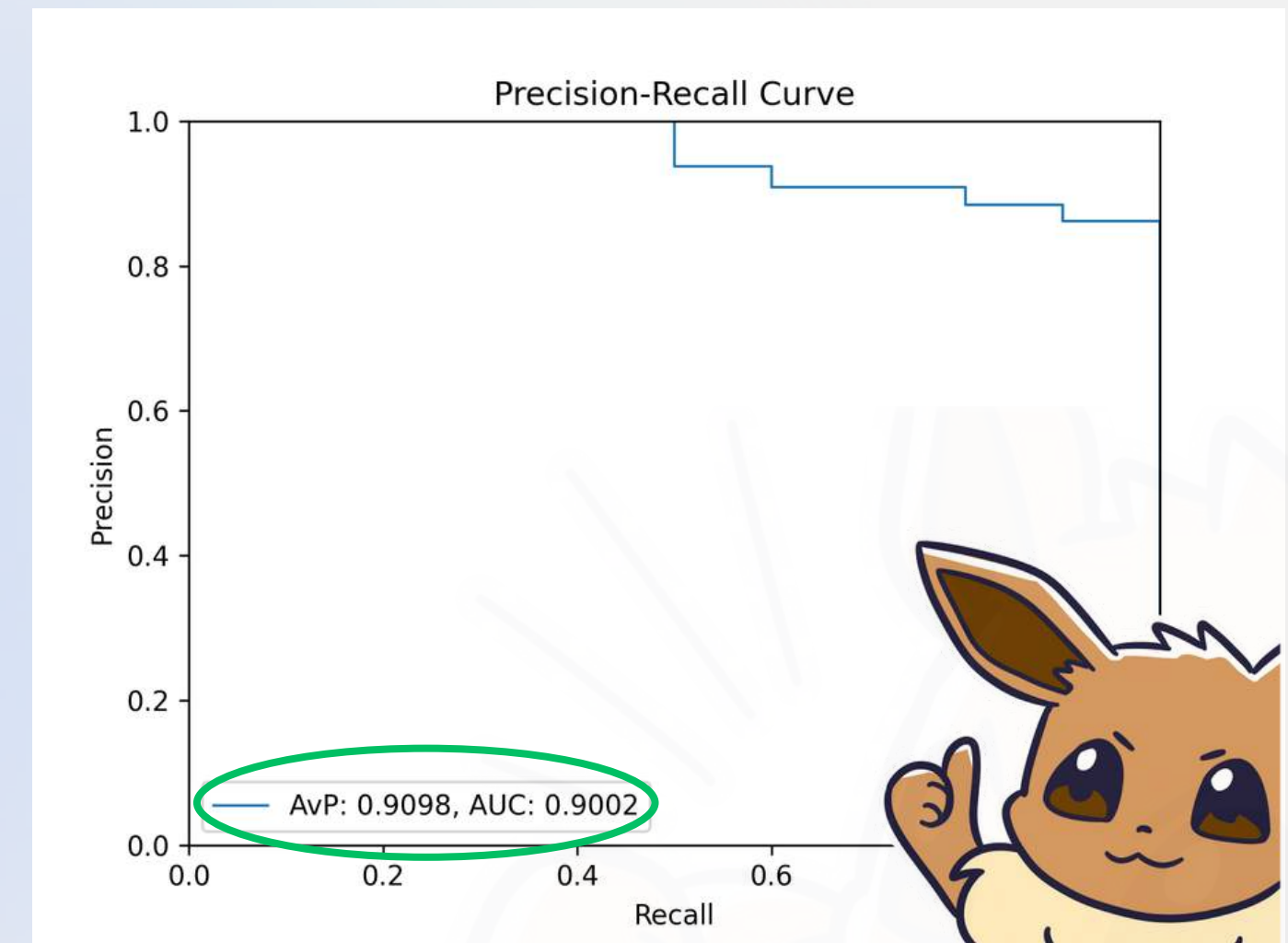
We made several evaluations with different metrics for each of the queries we made, on different search systems that we built to assess their quality.

Ash group composition



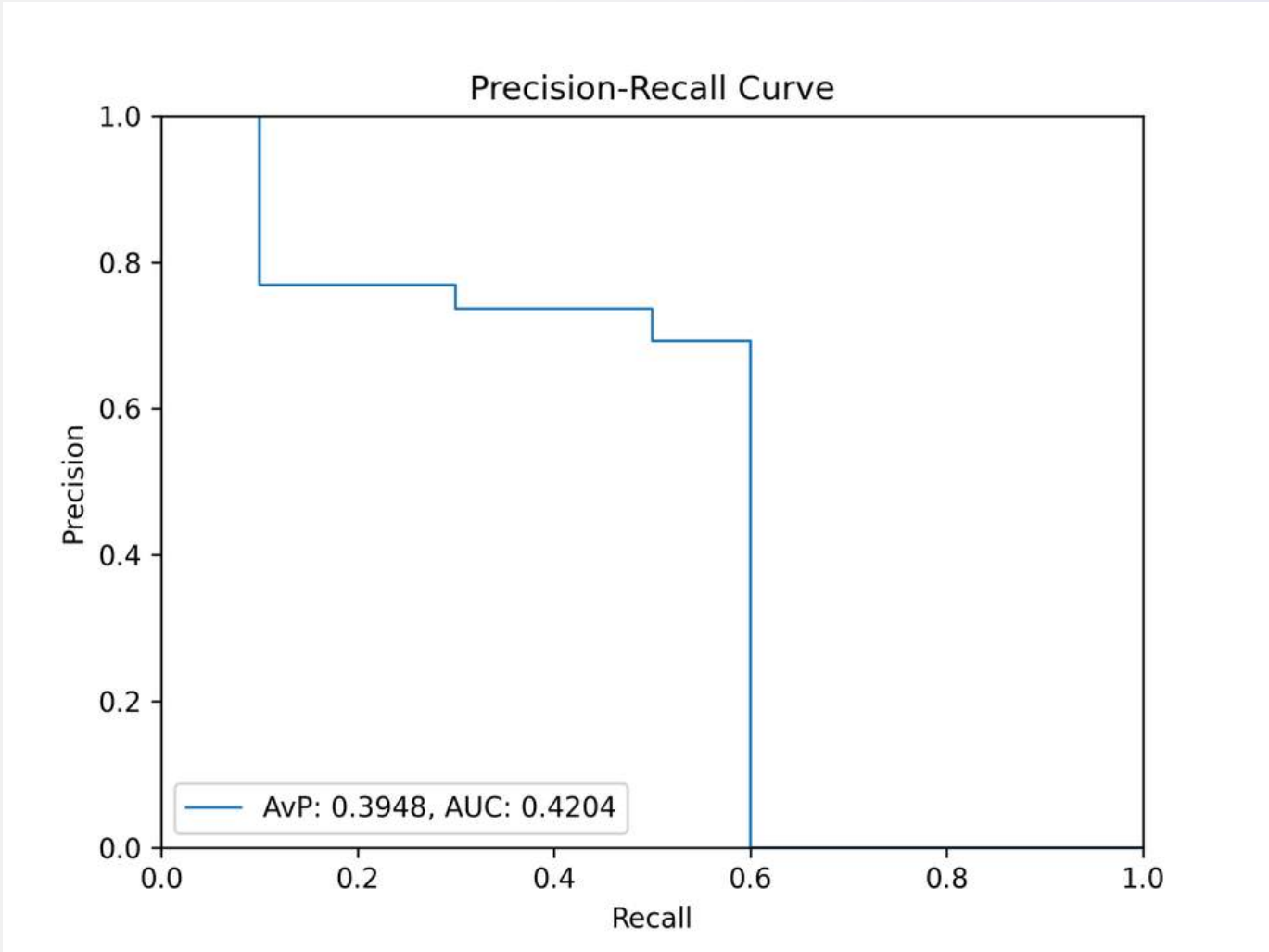
Simple system

VS



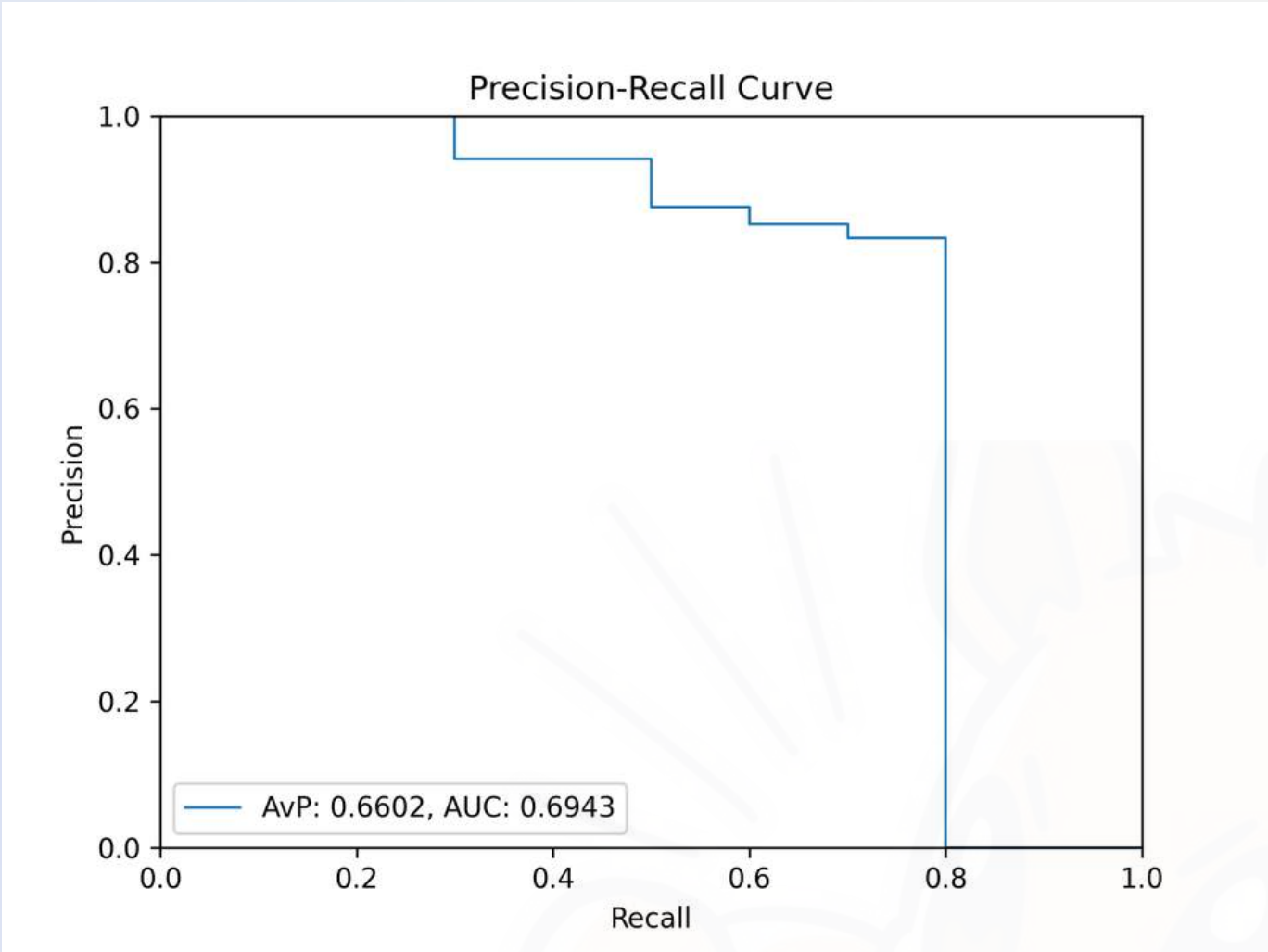
Complex system

Stealing pokemon



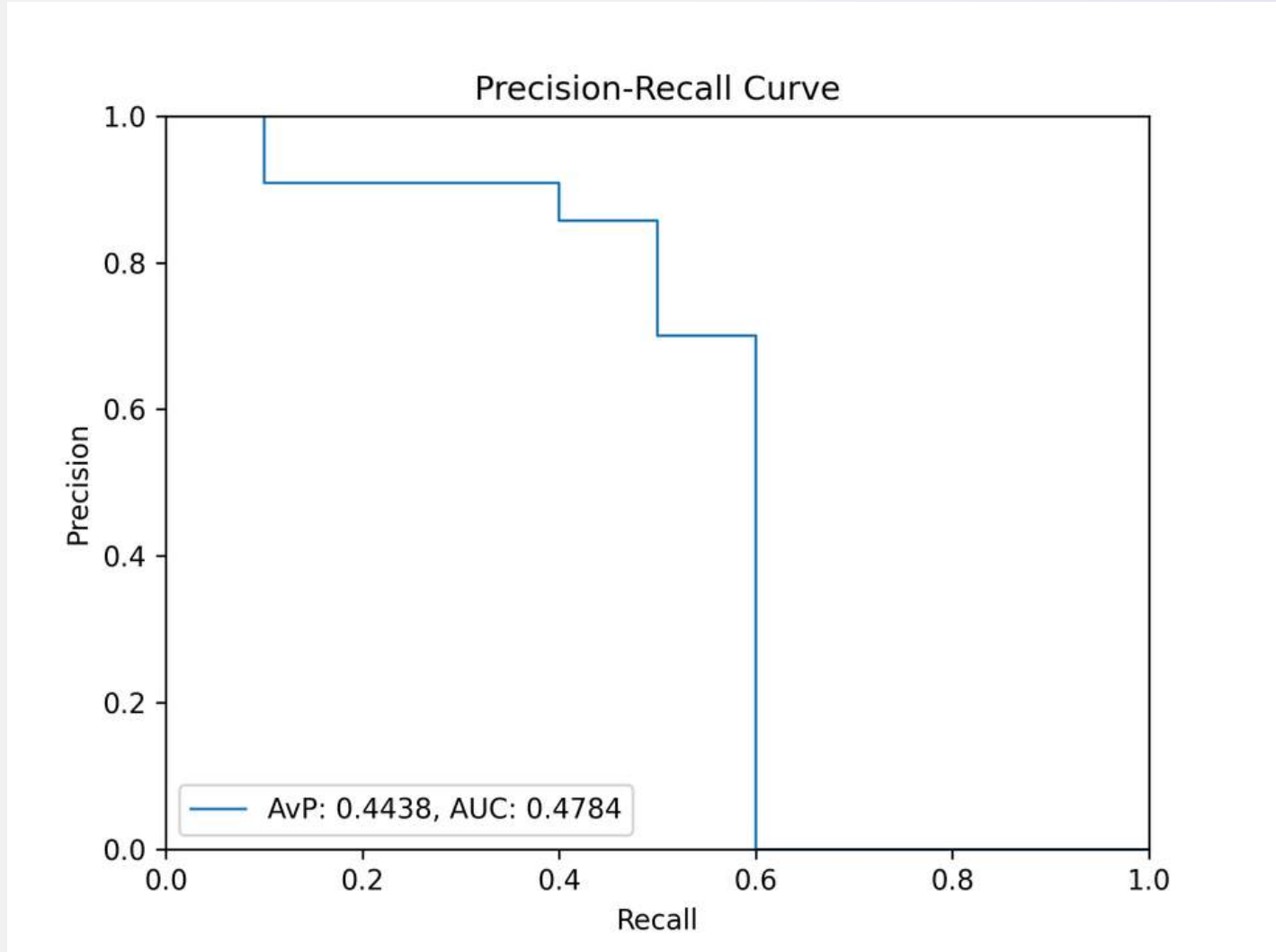
Simple system

VS



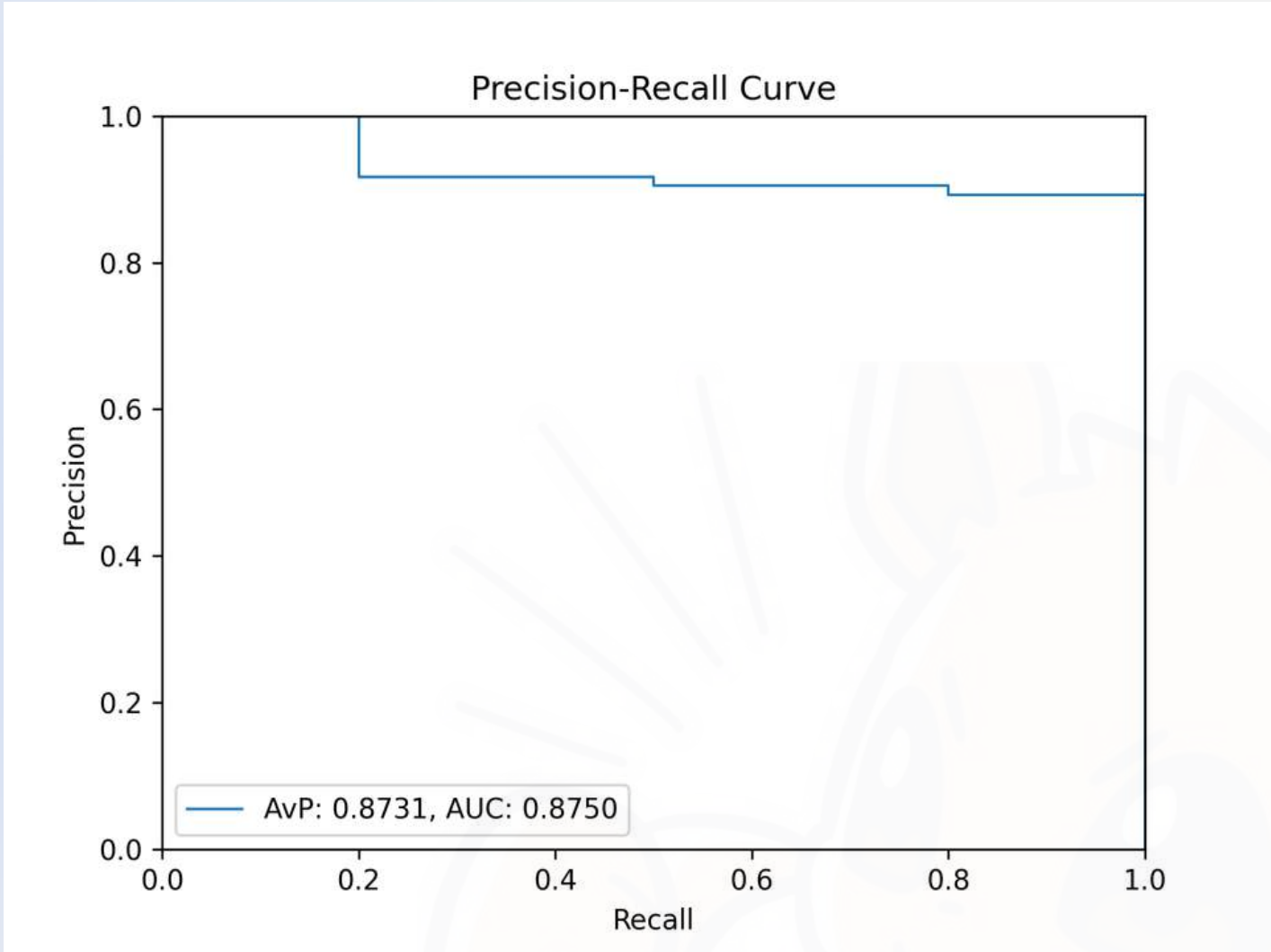
Complex system

Pokémon mega evolution



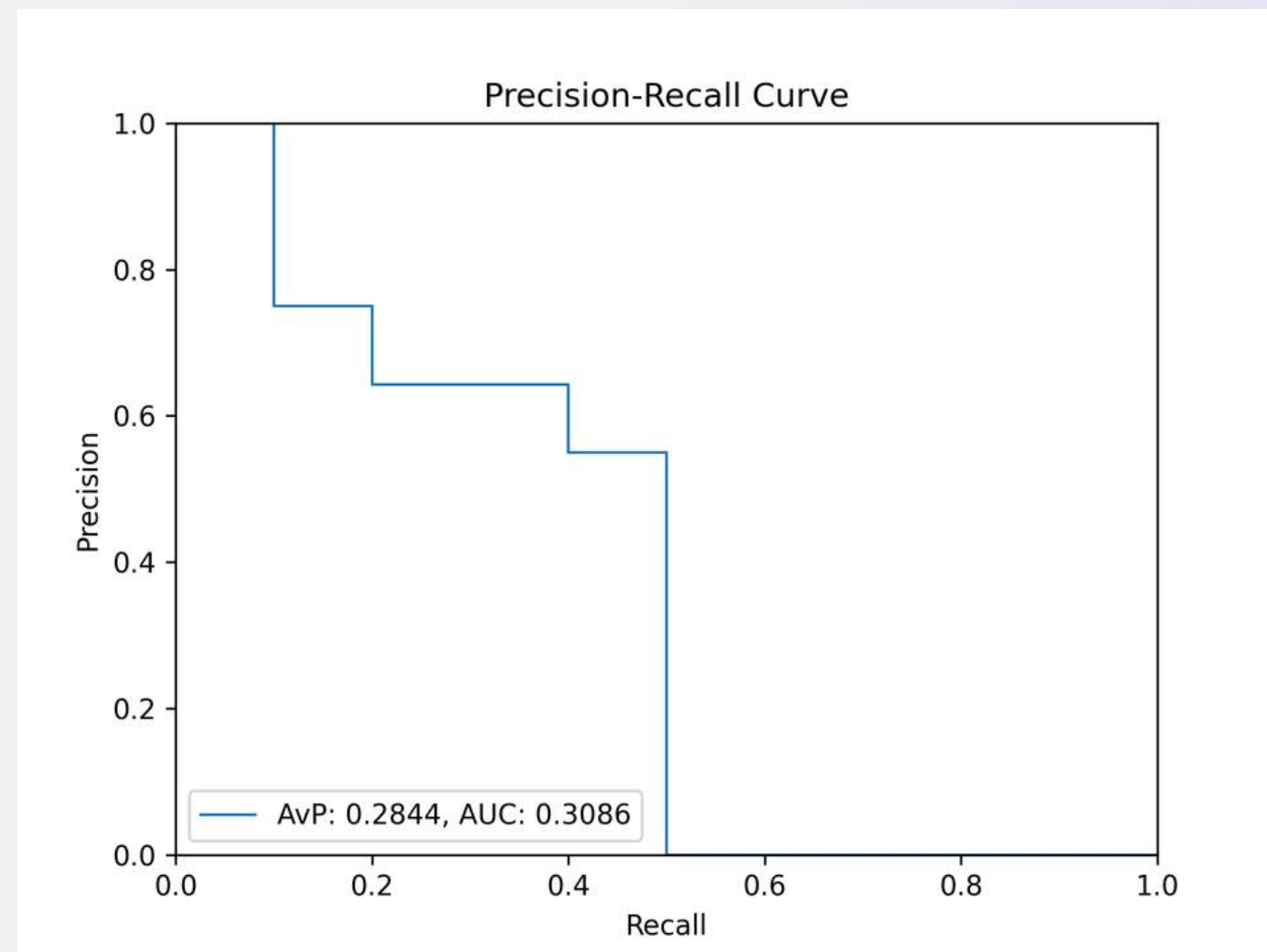
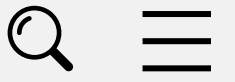
Simple system

VS



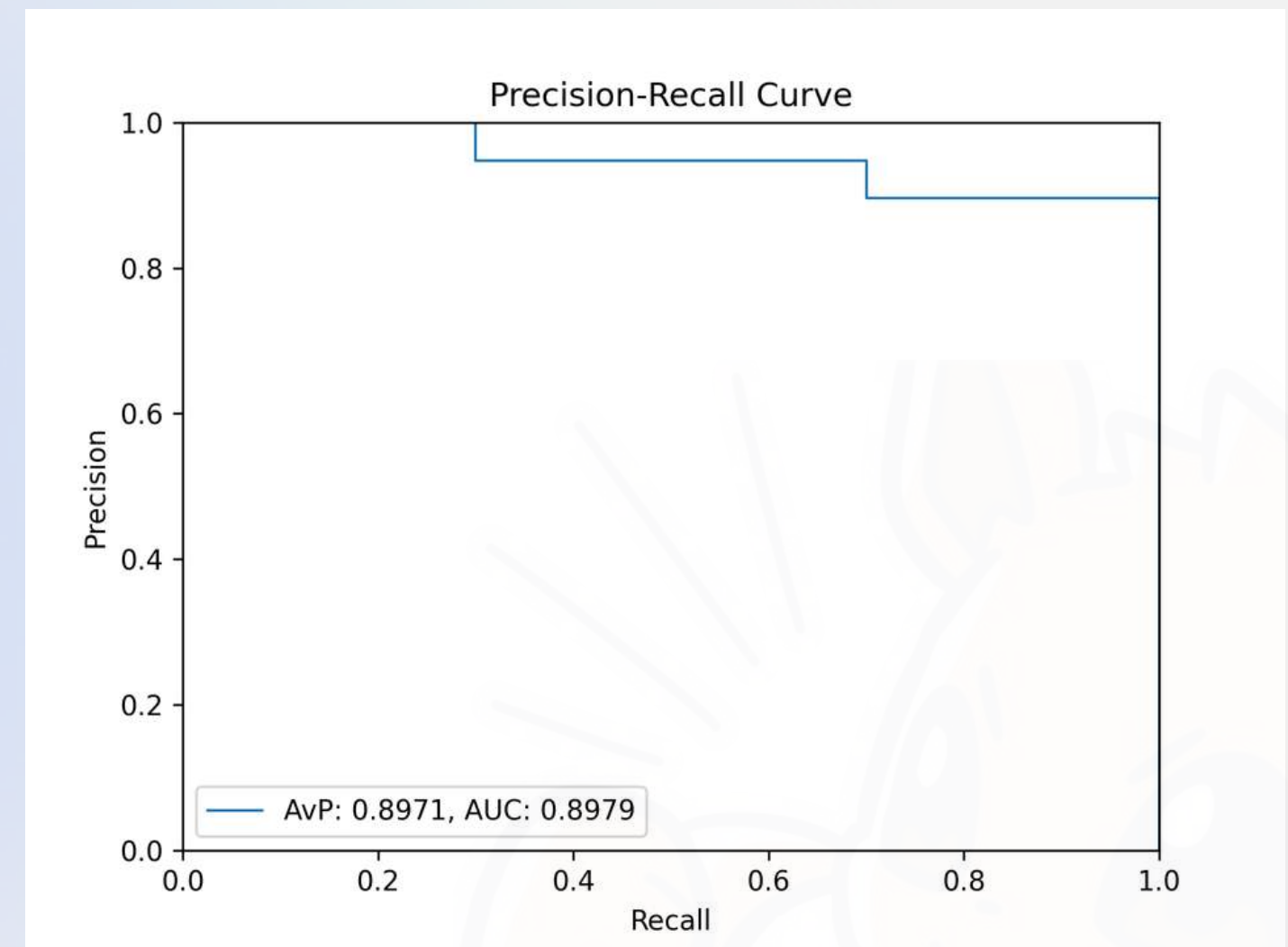
Complex system

Ash winning gym battles



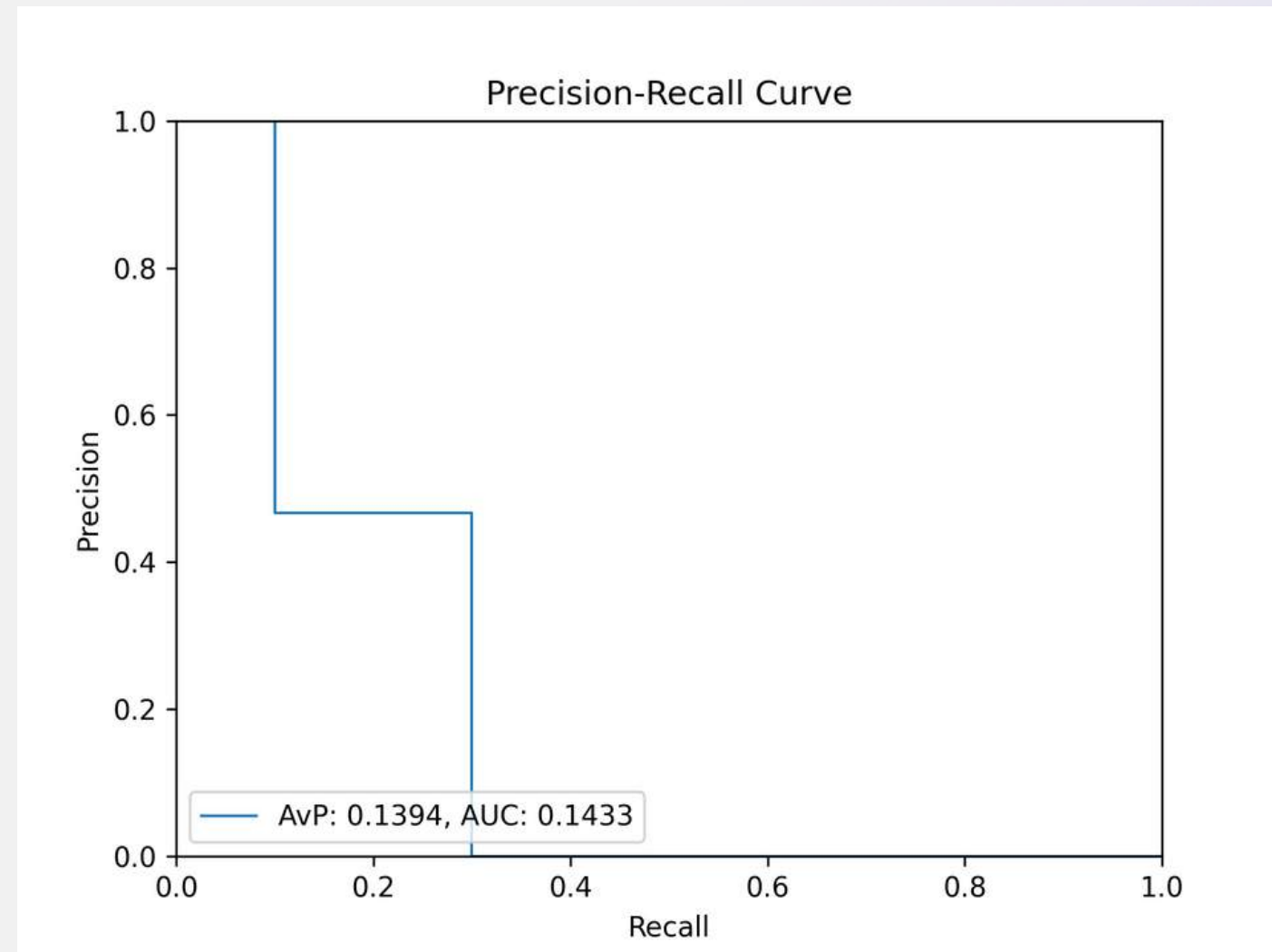
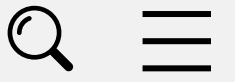
Simple system

VS



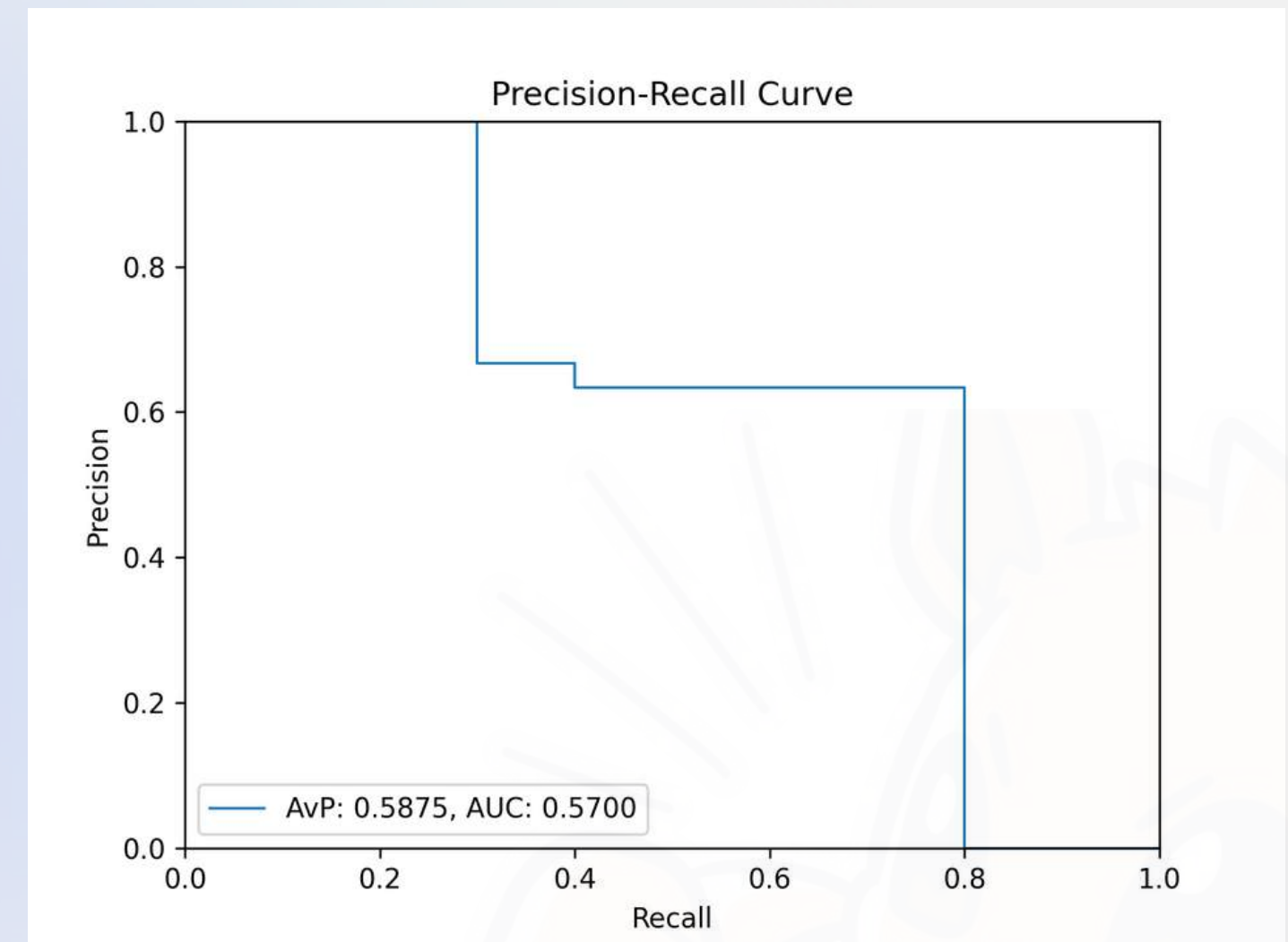
Complex system

Angry Pokémon

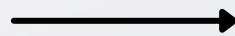


Simple system

VS



Complex system



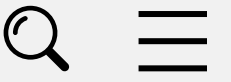
Future Work

Enhance synonyms support

Optimize nested documents

Query parameters fine-tuning





References



[1] [Solr](#)

[2] [Solr Guide](#)